Avrupa Birliği tarafından finanse edilmektedir

5. Gün: Modüler Kod ve Fonksiyonlar

Yazılım Geliştirme Eğitimi (Seviye 2)



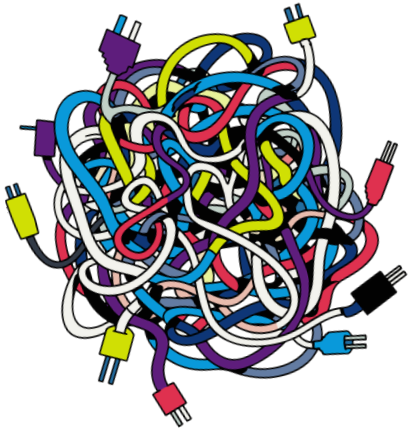
Python'da tekrar eden kodları nasıl önleriz?

Bölüm 1

Neden Fonksiyon Kullanmalıyız?

Bugüne kadar yazdığımız kodlar genellikle yukarıdan aşağıya doğru bir kez çalışıp bitiyordu. Ancak gerçek dünyada aynı işlemi defalarca yapmamız gerekir.

- **Spagetti Koddan Kurtulun:** Aynı kodu 10 farklı yere kopyala-yapıştır yaparsanız, bir hata çıktığında 10 yeri de düzeltmeniz gerekir!
- **Modüler Mimari:** Kodumuzu küçük, anlamlı parçalara böleriz. Bu parçalara "Fonksiyon" diyoruz.
- **Yeniden Kullanılabilirlik (Reusability):** Bir kere yaz, istediğin kadar çalıştır.





Kavramlar

Fonksiyon Nasıl Tanımlanır? (def)

Python'da kendi fonksiyonumuzu yaratmak için def (define - tanımla) anahtar kelimesini kullanırız.

Temel Kural:

1. Önce def yaz.
2. Fonksiyonuna anlamlı bir isim ver.
3. Parantez aç kapa () ve iki nokta üst üste : koy.
4. Bir alt satıra geçip, kodlarını girintili (indentation) olarak yaz.

```
# Örnek Şablon:
def fonksiyon_ismi():
    # Burası fonksiyonun içi (girintiye dikkat!)
    print("Fonksiyon çalıştı!")
```

Birlikte Yazıyoruz

İlk Fonksiyonumuzu Yazalım!

IDE'lerinizi açın. Şimdi sadece çağrıldığında ekrana "Merhaba Yazılımcı!" yazdıran basit bir fonksiyon tanımlayacağız.

```
# 1. Adım: Fonksiyonu tanımlıyoruz (Şu an hiçbir şey yazmaz, sadece öğretiriz)
def selam_ver():
    print("Merhaba Yazılımcı!")
    print("Bugün harika işler başaracağız.")

# 2. Adım: Fonksiyonu çağırıyoruz (Kullanıyoruz)
# Eğer çağırmazsanız, Python bu fonksiyonu bilir ama çalıştırmaz.
selam_ver()
selam_ver() # İsterseniz 100 kere çağırabilirsiniz!
```

Ne oldu? Kodu bir kez yazdık, ama `selam_ver()` diyerek iki kez çalıştırdık.

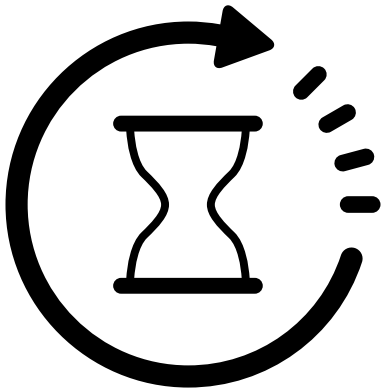
💻 Sıra Sende 1

Favori Yazılım Dilini Söyle!

Şimdi klavyeler sizde! Sadece 2 dakikanız var.

- **Görev:** favori_dil adında bir fonksiyon oluşturun.
- Bu fonksiyon çağrıldığında ekrana şunu yazdırmalı:

  "Benim favori yazılım dilim Python!"
- Fonksiyonu tanımladıktan sonra, programın sonunda **3 defa** peş peşe çağırın (çalıştırın).



Not: Hata yapmaktan korkmayın. Hatalar en iyi öğretmendir. Çözüm bir sonraki slaytta!

✅ Çözüm: Sıra Sende 1

Favori Yazılım Dili - Çözümü

Kimler doğru sonuca ulaştı? İşte kodumuzun olması gereken hali:

```
# Fonksiyonu tanımlama (Blueprint/Plan aşaması)
def favori_dil():
    print("Benim favori yazılım dilim Python!")

# Fonksiyonu çalıştırma aşaması (3 kez çağırıyoruz)
favori_dil()
favori_dil()
favori_dil()
```

Eğer hata aldıysanız, def sonrasındaki iki noktayı (`:`) veya `print` öncesindeki boşluğu (indentation) kontrol edin.

Bölüm 2

Parametre Alan Fonksiyonlar

Fonksiyonlarımız hep aynı şeyi mi yapacak? Hayır! Onlara dışarıdan bilgi gönderebiliriz. Bu bilgilere **Parametre** veya **Argüman** diyoruz.

Fonksiyonu bir fırın olarak düşünün; içine hangi malzemeyi (parametre) koyarsanız, ona göre bir ürün çıkarır.

```
# 'isim' burada bir parametredir. Dışarıdan gelecek bilgiyi temsil eder.
def kisiye_ozel_selamla(isim):
    print(f"Hoş geldin, {isim}!")

# Çağırırken parantez içine değeri (argümanı) gönderiyoruz:
kisiye_ozel_selamla("Ahmet")  # Çıktı: Hoş geldin, Ahmet!
kisiye_ozel_selamla("Ayşe")   # Çıktı: Hoş geldin, Ayşe!
```

İleri Seviye

Çoklu Parametre Kullanımı

Bir fonksiyona sadece bir değil, istediğiniz kadar parametre gönderebilirsiniz. Sadece aralarına virgül koymanız yeterlidir.

```
# İki sayıyı toplayan bir fonksiyon
def toplama_yap(sayi1, sayi2):
    sonuc = sayi1 + sayi2
    print(f"İşlemin sonucu: {sonuc}")

# Değerleri sırasıyla gönderiyoruz:
toplama_yap(10, 5)   # sayi1=10, sayi2=5 olur. Sonuç: 15
toplama_yap(50, 20) # sayi1=50, sayi2=20 olur. Sonuç: 70
```

Parametreler sayesinde fonksiyonlarımız çok daha yetenekli ve dinamik hale geldi!

💻 Sıra Sende 2

Dikdörtgen Çevresi Hesaplayıcı

Şimdi matematik bilginizi ve kodlama yeteneğinizi birleştirin.

- **Görev:** cevre_hesapla adında bir fonksiyon yazın.
- Bu fonksiyon 2 parametre almalı: kisa_kenar ve `uzun_kenar`.
- Fonksiyon içinde dikdörtgenin çevresini hesaplayın: `(kısa+uzun) * 2`
- Hesaplanan sonucu ekrana yazdırın (print ile).
- Kodu yazdıktan sonra fonksiyonu 5 ve 10 değerleriyle çağırıp test edin.

✅ Çözüm: Sıra Sende 2

Dikdörtgen Çevresi - Çözüm

İşte yazdığınız kodun sağlaması:

```
def cevre_hesapla(kisa_kenar, uzun_kenar):
    # Matematiksel işlemi yapıyoruz
    cevre = (kisa_kenar + uzun_kenar) * 2

    # Sonucu ekrana basıyoruz
    print(f"Dikdörtgenin çevresi: {cevre} cm")

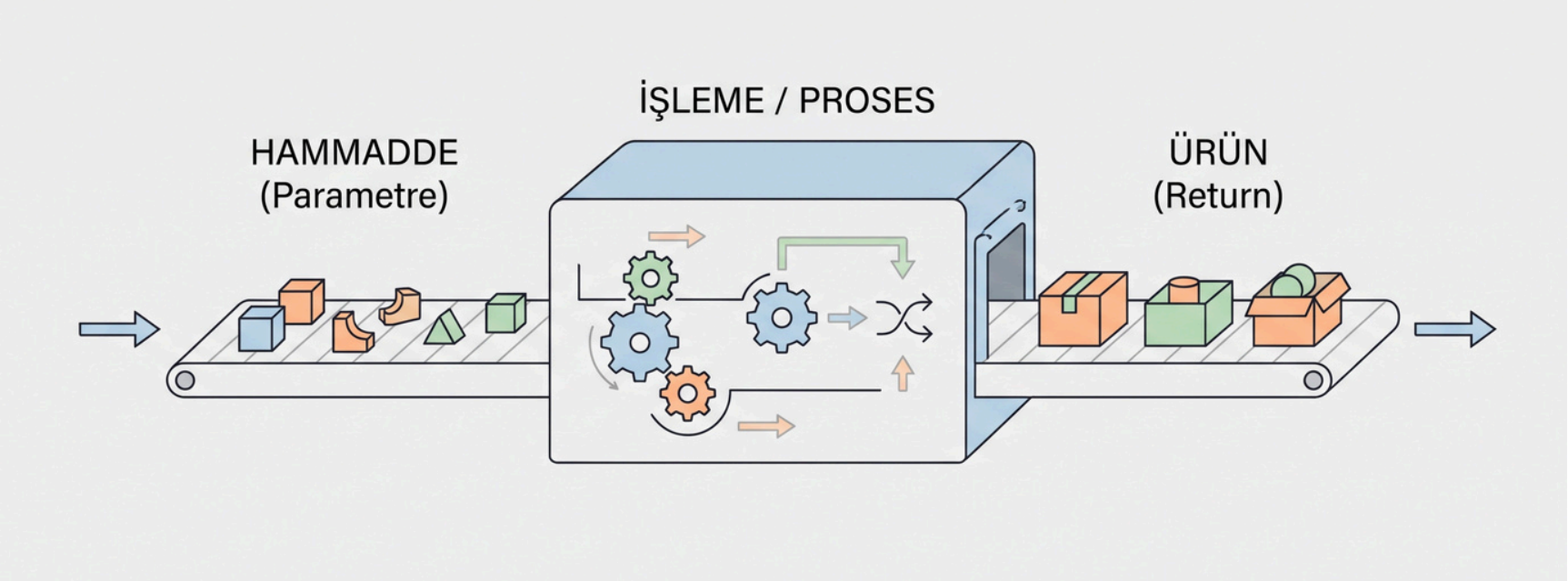
# Fonksiyonu test etme
cevre_hesapla(5,    10)# (5 + 10) * 2 = 30 yazdırır
cevre_hesapla(7, 12)  # (7 + 12) * 2 = 38 yazdırır
```

Dikkat ettiyseniz, işlemi bir değişkene (`cevre`) atamak, kodun okunabilirliğini artırdı.

Bölüm 3

Print vs Return (Değer Döndürme)

Şu ana kadar sonuçları hep print() ile sadece "ekranda gösterdik". Ama ya o sonuca programın başka bir yerinde ihtiyacımız olursa?



- print() sadece ekrana yazı yazar, değeri hafızada tutmaz. (Sadece gösteri içindir).
- return (döndür) ise; fonksiyonun işlediği veriyi, fonksiyonu çağıran yere "gerçek bir veri (değer)" olarak geri yollar.

Kavramlar

Return Kullanımı

Bir değer `return` edildiğinde, onu bir değişkene eşitleyebiliriz.

```
def carpma_yap(a, b):
    sonuc = a * b
    return sonuc  # Fonksiyon biter ve değeri dışarı atar.

# Fonksiyondan DÖNEN (return edilen) değeri x değişkenine hapsediyoruz!
x = carpma_yap(4, 5)

print(f"Veritabanına kaydedilecek değer: {x}")
# Artık x'in değeri 20 oldu. Onu istediğimiz gibi matematikte tekrar kullanabiliriz.
print(x + 100) # 120 yazar. Eğer return yerine print kullansaydık bunu yapamazdık!
```

💻 Sıra Sende 3

Sayı Çift mi? (Return Pratiği)

Bu çok önemli bir pratik, dikkatli yapalım:

- **Görev:** cift_mi adında bir fonksiyon yazın.
- Tek bir parametre alsın: sayi.
- Fonksiyon içinde kontrol edin: Eğer sayı 2'ye tam bölünüyorsa (`sayi % 2 == 0`), dışarıya `True` değerini **return** etsin.
- Değilse, False **return** etsin.
- **Dikkat:** Fonksiyon içinde ASLA print() kullanmayın.
- Çağırırken sonucu bir değişkene atayıp, o değişkeni print ile yazdırın.

✅ Çözüm: Sıra Sende 3

Sayı Çift mi? - Çözüm

İşte return kullanan mantıksal bir fonksiyon yapısı:

```
def cift_mi(sayi):
    if sayi % 2 == 0:
        return True
    else:
        return False

# Fonksiyondan dönen sonucu 'kontrol' değişkenine atıyoruz
kontrol1 = cift_mi(10)
kontrol2 = cift_mi(7)

print("10 çift mi?", kontrol1)  # True yazdırır
print("7 çift mi?", kontrol2)   # False yazdırır
```

Varsayılan (Default) Parametreler

Bazen kullanıcı bir parametre göndermeyi unutabilir veya biz bir parametrenin standart bir değeri olmasını isteyebiliriz.

Böyle durumlarda parametreye bir "varsayılan değer" (default value) atarız.

```
# Eğer 'mesaj' gönderilmezse, otomatik olarak 'Merhaba' değerini alacak.
def kullanici_olustur(isim, mesaj="Merhaba"):
    print(f"{mesaj}, {isim}!")

# 1. Durum: İki parametreyi de gönderiyoruz
kullanici_olustur("Ali", "Günaydın")  # Çıktı: Günaydın, Ali!

# 2. Durum: Sadece isim gönderiyoruz, mesaj varsayılan değeri kullanıyor
kullanici_olustur("Zeynep")          # Çıktı: Merhaba, Zeynep!
```

İsimlendirilmiş Argümanlar (Keyword Args)

Fonksiyonlara parametre gönderirken sırayı ezberlemek zorunda değilsiniz. Parametrelerin ismini açıkça belirterek gönderebilirsiniz. Bu, kodun çok daha okunabilir olmasını sağlar.

```
def profil_yarat(ad, yas, sehir):
    print(f"{ad}, {yas} yaşında ve {sehir}'de yaşıyor.")

# Normal (Sıralı) Kullanım:
profil_yarat("Murat", 25, "Ankara")

# İsimlendirilmiş Kullanım (Sıra önemsizdir!):
profil_yarat(sehir="İzmir", ad="Buse", yas=22)
# Çıktı: Buse, 22 yaşında ve İzmir'de yaşıyor.
```

Çok Önemli Kavram!

Kapsam (Scope) - Yerel vs Global Değişkenler

Bir fonksiyonun içinde tanımlanan değişkenler (Yerel/Local), **sadece o fonksiyonun içinde yaşar**. Dışarıdan onlara ulaşılamaz. Las Vegas kuralı gibidir: Fonksiyonun içinde olan, fonksiyonun içinde kalır.

```
def test_fonksiyonu():
    gizli_sifre = "12345"  # Bu bir YEREL değişkendir.
    print("Şifre oluşturuldu.")

test_fonksiyonu()

# Aşağıdaki satır HATA verecektir! (NameError)
# Çünkü gizli_sifre fonksiyon dışından görülmez.
print(gizli_sifre)
```

Bu yüzden fonksiyondaki bir veriyi dışarı almak için `return` kullanırız!

💻 Sıra Sende 4

Final Görevi: İndirim Hesaplayıcı!

Modüler kod mimarisine ilk adımımızı atıyoruz. Bu görevde hem parametre hem de return kullanacağız.

- **Görev:** indirim_uygula adında bir fonksiyon yazın.
- İki parametre alsın: fiyat ve indirim_orani. (İndirim oranının varsayılan (default) değeri 10 olsun).
- `Fonksiyon içinde yeni fiyatı hesaplayın: yeni_fiyat = fiyat - (fiyat * indirim_or / 100)`
- Hesaplanan bu yeni fiyatı return edin.
- Dışarıda fonksiyonu 1000 TL fiyat ve %20 indirimle çağırıp, dönen sonucu `print` ile yazdırın.

✅ Çözüm: Sıra Sende 4

İndirim Hesaplayıcı - Çözüm

Kapsamlı bir "gerçek dünya" fonksiyonu yazmış olduk:

```python
# indirim_orani'na varsayılan olarak 10 atadık
def indirim_uygula(fiyat, indirim_orani=10):
    indirim_miktari = fiyat * (indirim_orani / 100)
    guncel_fiyat = fiyat - indirim_miktari

    return guncel_fiyat

# 1. Özel indirimle çağırma
son_fiyat = indirim_uygula(1000, 20)
print(f"Özel indirimli fiyat: {son_fiyat} TL")  # 800.0 TL

# 2. Varsayılan indirimle (sadece fiyat girerek) çağırma
standart_fiyat = indirim_uygula(1000)
print(f"Standart indirimli fiyat: {standart_fiyat} TL") # 900.0 TL
```

Modüler Kod - Fonksiyonları İç İçe Kullanmak

Gerçek projelerde, yazdığımız fonksiyonlar başka fonksiyonları çağırarak büyük sistemleri oluştururlar. Tıpkı bir arabanın parçalarının birlikte çalışması gibi!

```
def kdv_ekle(fiyat):
    return fiyat * 1.20

def kargo_ekle(fiyat):
    return fiyat + 50

def siparis_tamamla(sepet_tutari):
    # Bir fonksiyonun içinde diğerlerini kullanıyoruz!
    tutar_kdvli = kdv_ekle(sepet_tutari)
    toplam_tutar = kargo_ekle(tutar_kdvli)
    print(f"Müşteriden Alınacak Toplam: {toplam_tutar} TL")

siparis_tamamla(100) # 100 -> KDV ile 120 -> Kargo ile 170 yazar.
```

Günün Özeti: Neler Başardık?

- ✔️ Spagetti kod yazmayı bıraktık, Modüler tasarıma geçtik.
- ✔️ def ile kendi kod bloklar ımızı oluşturduk.
- ✔️ Fonksiyonlara dışarıdan veri gönderdik (Parametreler/Argümanlar).
- ✔️ Fonksiyondan dışarıya veri aldık (return).
- ✔️ Varsayılan (default) ve isimlendirilmiş argümanları kullandık.
- ✔️ Global ve Yerel kapsam (Scope) mantığını anladık.

Haftaya Nesne Yönelimli Programlama (OOP) ile görüşmek üzere!